

DelosDB Storage Architecture

Unified I/O Abstraction · MVCC Concurrency · Clean Architecture

Non-Conservative Design Proposal · June 2026 · DelosDB Engineering

Sources: PostgreSQL · MariaDB/InnoDB · H2 · Apache Calcite · HerdDB · MapDB

Research: LeanStore VLDB 2024 · Alhomssi & Leis VLDB 2023 · Jasny et al. PVLDB 2026

.42, HerdDB, and MapDB, plus three peer-reviewed VLDB/PVLDB papers (2023–2026), and the JDK 21–25 platform roadmap. The architecture is org

1

Architecture — Four Independent Layers

The architecture separates DelosDB's storage concern into four non-overlapping layers. Each layer has exactly one responsibility, depends only on the layer below it, and exposes a narrow Java interface upward. No layer imports implementation classes from another layer. This is the lesson from all five production systems studied: every system that violated this rule (Derby's monolithic storage, MariaDB's 218-method handler, PostgreSQL's `t_xmin` baked into the physical tuple) created decades of maintenance debt. Every system that honored it (Calcite's capability-segregated Table interfaces, MapDB's Volume hierarchy, HerdDB's `DataStorageManager`) remained extensible without surgery.

SQL EXECUTION LAYER

- Derby parser · binder · optimizer · AsmJava activations
- DelosTableScanResultSet · DelosInsertResultSet · DelosDeleteResultSet · DelosUpdateResultSet
- DelosTableScanProviderLookup · GenericResultSetFactory branch
- DelosVersionedStorageQueryTreeClassifier (JavaCC parse-tree, no regex)

Depends on: Contract Layer only

CONTRACT LAYER (Calcite-inspired capability segregation)

- DelosTableAccess · DelosFilterableTableAccess · DelosIndexableTableAccess
- DelosMutableTableAccess · DelosCostableTableAccess
- DelosAccessContext (typed key-lookup, Derby ContextManager idiom)
- DelosTableCapability (structural) · DelosTableGuarantee (semantic promises)
- DelosRowIdentity (opaque, no cross-provider equality) · DelosPredicate · DelosScan

Depends on: MVCC Layer interfaces only

MVCC CONCURRENCY LAYER

- MvccSnapshot · MvccVisibility · MvccVersionChain · MvccTransactionCatalog
- MvccCapturedVisibility · MvccStatementSnapshot (command-sequence visibility)
- MvccHistoryPrunedException (distinguishable from 'not visible' — InnoDB lesson)
- MvccVacuum · MvccVacuumWatermark · MvccTransactionOutcomeLog
- EngineMvccTableAccess (implements all capability interfaces)

Depends on: I/O Layer interfaces only

I/O LAYER (DelosPageVolume hierarchy)

- DelosPageVolume interface — 6 methods, ReentrantLock, SyncPolicy enum
- FileChannelPageVolume · MappedPageVolume · OffHeapPageVolume
- FaultInjectingPageVolume · ODirectPageVolume (JDK 10+) · IoUringPageVolume (JDK 22+)
- DelosVolumeFactory — matched pairs (data + mutation-log + outcome-log + index)
- MvccPage using MemorySegment+Arena (JDK 22) replacing ByteBuffer

Depends on: JDK APIs only — no DelosDB imports

2 I/O Layer — DelosPageVolume Full Design

The I/O layer has exactly one job: move pages between memory and durable storage. It knows nothing about MVCC, SQL, or row formats. Every implementation decision is independently verifiable from the five Apache 2.0 sources studied.

2.1 The Interface — verified against MapDB Volume, HerdDB ManagedFile

```
public interface DelosPageVolume extends AutoCloseable {

    // Core page operations – O(1) by page ID (MapDB: array index, not TreeMap)
    MvccPage readPage(MvccPageId id)           throws IOException;
    void      writePage(MvccPage page)          throws IOException;
    MvccPage allocatePage(int pageType)        throws IOException;
    long      pageCount()                      throws IOException;

    // Durability boundary – pluggable (HerdDB ManagedFile.sync(boolean metaData) pattern)
    void      force()                          throws IOException;
    SyncPolicy syncPolicy();

    @Override void close() throws IOException;

    enum SyncPolicy {
        FULL,           // channel.force(true) – production default
        METADATA_ONLY, // channel.force(false) – metadata only
        NONE            // no-op – OffHeapPageVolume, FaultInjectingPageVolume
    }
}
```

MvccPage moves from ByteBuffer (2 GB ceiling, GC-managed) to MemorySegment+Arena (JDK 22, 64-bit long addressing, deterministic off-heap deallocation). All six implementations below operate identically from the caller's perspective — the switch is a construction-time parameter, not a runtime branch.

2.2 Six Implementations

Implementation	Backing store	force()	Source / License	Status
FileChannelPageVolume	FileChannel + ReentrantLock (replaces synchronized)	channel.force(FULL/META)	MapDB FileChannelVol Apache 2.0	NOW
MappedPageVolume	FileChannel.map() chunked MemorySegment+Arena	segment.force() per chunk	MapDB MappedFileVol Apache 2.0	NEXT
OffHeapPageVolume	MemorySegment[] indexed by page ID (O(1))	no-op (NONE)	MapDB MemoryVol pattern Apache 2.0	NEXT
FaultInjectingPageVolume	Delegates to any above FaultSchedule wrapper	configurable failure	Original — crash-recovery proof harness	NEXT
ODirectPageVolume	FileChannel + ExtendedOpenOption.DIRECT	none (O_DIRECT = fsync-free)	HerdDB ODirectFileOutputStream Apache 2.0	FUTURE

Implementation	Backing store	force()	Source / License	Status
ioUringPageVolume	FFM Linker → liburing io_uring SQ/CQ rings	IORING_OP_FSYNC async	JEP 454 (JDK 22) Jasny PVLDB 2026	FUTURE

2.3 DelosVolumeFactory — matched pairs

Each durable structure (page file, mutation log, transaction outcome log, index) has a different I/O access pattern and must not be forced into one interface. A factory provides matched pairs so all four structures for one provider are created consistently — the same backing mode (file / mmap / off-heap) for all four, or deliberately mixed for testing. Modeled on MapDB's Volume.Factory pattern (Apache 2.0).

```
public interface DelosVolumeFactory {
    DelosPageVolume openPageVolume(Path path)          throws IOException;
    DelosAppendVolume openMutationLog(Path path)        throws IOException; // append-only
    DelosAppendVolume openOutcomeLog(Path path)         throws IOException; // append-only
    DelosPageVolume openIndexVolume(Path path)          throws IOException;

    static DelosVolumeFactory fileChannel(SyncPolicy policy) { ... }
    static DelosVolumeFactory offHeap()                    { ... } // tests
    static DelosVolumeFactory faultInjecting(FaultSchedule f) { ... } // crash proofs
}
```

2.4 Critical JDK 21 fix — ReentrantLock everywhere

MvccPageFile currently has 7 synchronized methods. On JDK 21–23 every one pins a virtual thread's carrier OS thread for the full I/O duration (JEP 444). JEP 491 (JDK 24) fixes most cases but cannot unpin native calls — and `channel.force(true)` is a native syscall. The fix: `ReentrantLock` throughout, exactly as MapDB's `ByteBufferVol.growLock` already demonstrates (Apache 2.0, borrowable). This is a one-day change with no behavioral impact. It must happen before any other I/O work.

3

MVCC Concurrency Layer — Current & Future

3.1 What is already implemented and verified

The following was verified line-by-line against PostgreSQL `proccarray.c`, H2 `TransactionMap.java/TxDecisionMaker.java`, and InnoDB `row0vers.cc` during this session. It is real, not aspirational:

- True snapshot isolation: active-at-capture transaction exclusion (verified correct against PostgreSQL `GetSnapshotData()` xip[] logic)
- Command-sequence visibility: `MvccStatementSnapshot` / `MvccCommandSequence` — own-statement visibility within a transaction (PostgreSQL `Cmin/Cmax` lesson, A46/A47)
- Distinguishable history-pruned failure: `MvccHistoryPrunedException` separate from 'not visible' (InnoDB `DB_MISSING_HISTORY` lesson, A44)
- Vacuum watermark: `MvccVacuumWatermark` prevents premature version pruning while long-running queries are active (Alhomssi & Leis VLDB 2023, Section 3)
- Write-conflict mapping: `DelosMutationConflictMapper` → `SQLState 40001` (serialization failure, not deadlock — Javadoc note added)
- Durable mutation log: `MvccPageMutationLog` with torn-write protection
- Transaction outcome log: `MvccTransactionOutcomeLog` for unresolved outcome recovery
- Cost surface: `DelosCostableTableAccess` + `EngineMvccTableAccess.estimateTableCost()` → `VersionedStorageExecutionBridge.stats()`

3.2 Three gaps from research — named, scoped, sequenced

Graveyard Index

Alhomssi & Leis VLDB 2023, Section 4.1

Logically-deleted tuples remain in the main version chain, forcing `MvccVisibility.isVisible()` on every tombstone during a scan. The Graveyard Index moves tombstones to a side structure, making main-index scans skip deleted rows without visibility evaluation. Prerequisite: multi-concurrent benchmark demonstrating chain-length cost.

Adaptive version storage

Alhomssi & Leis VLDB 2023, Section 4.3

`MvccVersionChain` uses a fixed `List` for all tuples. Hot tuples (frequently updated) and cold tuples (rarely updated) should use different physical formats to minimize GC pressure and chain traversal cost.

`MvccVersionChain.deadVersionEstimate()` already tracks the data needed to classify hot vs cold.

Scalable commit protocol

Alhomssi & Leis VLDB 2023, Section 3

`MvccTransactionCatalog` commit path involves catalog mutation under lock. LeanStore's commit protocol avoids touching the write set on commit, minimizing cross-thread communication. `MvccCapturedVisibility` (A51) is already a step in this direction; the LeanStore protocol completes it.

3.3 Row-level locking — forced decision before code

	Option A — Honest conflict detection	Option B — Real row reservation
Shape	<code>validateMutable(context, rowIdentity)</code> or <code>prepareMutation(...)</code>	<code>tryLock(context, rowIdentity, mode)</code> → <code>DelosLockResult</code>
Semantics	Optimistic: detect conflict when row identity is already mutated	Pessimistic: reserve row before mutation, block concurrent writers
New infra	None — uses existing write-conflict detection	Lock table, transaction-owned lifecycle, release on commit/rollback
Guarantee	<code>SNAPSHOT_ISOLATION</code> yes <code>ROW_LOCKING</code> no	<code>SNAPSHOT_ISOLATION</code> yes <code>ROW_LOCKING</code> yes (real)
Timeline	2–3 weeks	6–10 weeks
Prerequisites	K1 answer, E1 guarantee flags already done	Option A first, then new infrastructure

4

Contract Layer — Capability-Segregated Interfaces

Modeled on Apache Calcite 1.42's `Table/FilterableTable/ModifiableTable` hierarchy (verified from source). Four sibling interfaces, not a ladder. Each engine implements only what it actually supports. Storeless is the negative proof — it implements only `DelosTableAccess`, proving no interface secretly requires storage.

Interface	Purpose	heap	delos_mvcc	storeless
<code>DelosTableAccess</code>	Identity + row shape + capabilities + guarantees	✓	✓	✓
<code>DelosFilterableTableAccess</code>	Scan with mutable filter-list + projection (Calcite <code>FilterableTable</code> lesson)	✓	✓	—
<code>DelosIndexableTableAccess</code>	Provider-owned index access	✓	✓	—
<code>DelosMutableTableAccess</code>	Insert/update/delete by row identity (NOT by SQL text)	✓	✓	—
<code>DelosCostableTableAccess</code>	Optional — scan + lookup cost estimates (separate, not on <code>FilterableTableAccess</code>)	proof	real	—

The mutable filter-list mechanism (Calcite's key insight, verified in `CsvFilterableTable.java`): the adapter receives a mutable `List<DelosPredicate>` and removes only predicates it can push down. Unpushed predicates remain for the caller to apply above the scan. This is the concrete mechanism that replaces regex-per-statement-type — one `scan()` method handles all predicate shapes, declaring honestly which it can optimize.

DelosAccessContext uses typed capability-key lookup (Derby's own `ContextManager` idiom: `getContextOrNull(CONTEXT_ID)`). heap adapter requires `TransactionController` via a heap-owned key. MVCC adapter requires `TxContext` via an MVCC-owned key. Neither key type appears in engine-kernel contracts.

5 SQL Execution Layer — Native Derby Path

5.1 What is working today (F0–F8, G0–G6 complete)

The following SQL shapes reach `delos_mvcc` tables through Derby's real parser/binder/compiler/AsmJava activation path — no regex interception, no `EmbedStatement` bridge — verified by F8 native-mode bridge bypass:

- `CREATE TABLE ... USING delos_mvcc` (F1: `sqlgrammar.jj storageProviderClause()`)
- Provider name persisted in `SYSTABLES` (`SYSTABLES_COLUMN_COUNT` = 6, F2)
- `SELECT * FROM T WHERE id = literal` (F4: `Qualifier[]` → `DelosPredicate`, column ordinal+1)
- `SELECT * FROM T WHERE id > literal` (G1: negated operator normalization)
- `SELECT * FROM T WHERE id BETWEEN a AND b` (G2)
- `SELECT * FROM T` (G3: full scan via `DelosTableScanResultSet.allowFullScan`)
- `SELECT COUNT(*) FROM T` (G4: Derby aggregate over Delos rows)
- `INSERT INTO T VALUES (...)` (F5: `DelosInsertResultSet`)
- `DELETE FROM T WHERE id = literal` (F6: scan → `DelosRowIdentity` → delete)
- `UPDATE T SET col = v WHERE id = literal` (F7: compact + full after-image handling)
- `CREATE INDEX ... ON T` (G5: `DelosCreateIndexProviderLookup` via `TableDescriptor`)

5.2 SQL coverage roadmap

SQL Shape	Mechanism	Status
<code>WHERE col1=v1 AND col2=v2</code>	Multi-predicate AND qualifier fix (<code>term>0 && length>1</code> → <code>length>1</code>)	FIX NOW
<code>WHERE col IN (...)</code>	<code>QueryTreeNode InListOperatorNode</code> → <code>DelosPredicate</code> list	L3 next lane
<code>WHERE col IS NULL / IS NOT NULL</code>	<code>QueryTreeNode IsNullNode</code> → <code>DelosPredicate</code>	L3 next lane
<code>ORDER BY col</code>	<code>QueryTreeNode OrderByList</code> → scan hint to provider	L3 next lane
<code>JOIN</code> (two <code>delos_mvcc</code> tables)	Nested-loop via two <code>DelosTableScanResultSet</code> scans	Future
<code>SELECT FOR UPDATE</code>	E4 Option B real row reservation	Future
<code>EXPLAIN</code> (MVCC path)	Research-facing observe surface	Future

6 Provider Parity — Route B → Destination A

Destination: heap and `delos_mvcc` are both live Delos providers under one contract. Route: keep heap Derby-native while `delos_mvcc` hardens the contract. This is the MariaDB model — multiple engines under one SQL surface, each declaring honest guarantees, no pretense of uniformity.

Gate	Question	Decision
K1 (probe only)	Can GenericResultSetFactory branch safely for default-provider heap? Can EngineHeapTableAccess.scan() honestly call ScanController? Can heap RowLocation fit DelosRowIdentity without lying? Does L1 row-lock contract need to be designed for two providers?	Binary A/B answer. No code change. 3–5 days investigation.
K1 = A (clean seam)	Heap seam is safe. GenericResultSetFactory can branch without touching FromBaseTable.generate().	→ L4 optimizer consumption → L3 broader SQL → L1 row reservation (designed for 2 providers) → A: heap live provider
K1 = B (too invasive)	Heap seam requires touching FromBaseTable.generate() or AsmJava emission. Too invasive now.	→ L4 optimizer consumption → L3 broader SQL → L1 MVCC-only row reservation → A deferred
Provider dispatch	DelosStorageDispatch (upper VFS layer, PostgreSQL smgr-inspired): 8 callbacks: identity, open, close, vacuum, recover, cost (Not 218 like MariaDB handler — MariaDB is the anti-pattern)	Built ONLY after two live providers exist. PostgreSQL smgrsw[] = one entry for decades. Don't repeat that.

7

Complete Implementation Sequence

#	Item	Layer	JDK	Weeks	Status
I-1	ReentrantLock in MvccPageFile (7 methods)	I/O	21	<1	FIX NOW
I-2	Remove lockAcquired from DelosMutationPreparation	Contract	21	<0.1	FIX NOW
I-3	DEADLOCK comment in DelosMutationConflictMapper	Contract	21	<0.1	FIX NOW
I-4	AND-conjunction fix in DelosTableScanResultSet	SQL	21	<0.5	FIX NOW
N-1	DelosPageVolume interface + 4 implementations (File, OffHeap, FaultInjecting, Mapped)	I/O	21/22	1–2	NEXT
N-2	MvccPage: ByteBuffer → MemorySegment + Arena	I/O	22	1	NEXT
N-3	DelosVolumeFactory matched-pair pattern	I/O	21	0.5	NEXT
N-4	K1 heap feasibility probe (investigation only)	All	21	0.5	NEXT
N-5	L4 optimizer cost consumption (H1–H4 already built)	SQL	21	1–2	NEXT
N-6	L3 broader SQL: AND-multi, IN, IS NULL, ORDER BY	SQL	21	2–4	NEXT
F-1	ODirectPageVolume (ExtendedOpenOption.DIRECT)	I/O	10+	1	FUTURE
F-2	E4 Option A: validateMutable / prepareMutation	MVCC	21	2–3	FUTURE
F-3	E4 Option B: tryLock + real reservation state	MVCC	21	6–10	FUTURE
F-4	Graveyard Index (Alhomssi & Leis VLDB 2023)	MVCC	21	4–6	FUTURE
F-5	Adaptive version storage (Alhomssi & Leis VLDB 2023)	MVCC	21	4–6	FUTURE
F-6	Scalable commit protocol (Alhomssi & Leis VLDB 2023)	MVCC	21	3–5	FUTURE
F-7	DelosStorageDispatch (post K1 + heap live)	All	21	2–3	FUTURE
F-8	IoUringPageVolume (FFM Linker + liburing)	I/O	22+	3–5	FUTURE
F-9	Calcite as MVCC SQL frontend (FilterableTable adapter)	SQL	21	6–10	FUTURE
F-10	Calcite Volcano optimizer (rule-based architecture)	SQL	21	10–20	LONG-TERM

8

License Compatibility — What Can Be Borrowed

System	License	Borrow code?	What to take
HerdDB	Apache 2.0	YES — directly	ManagedFile (fsync abstraction), ODirectFileOutputStream, DataStorageManager abstract class pattern
MapDB	Apache 2.0	YES — directly	Volume.ByteBufferVol.growLock (ReentrantLock pattern), MemoryVol structure, MappedFileVol.sync() per-chunk force
H2 Database	MPL 2.0 + EPL 1.0	NO — pattern only	OffHeapStore concept, FileStore<C> abstract class pattern. Write OffHeapPageVolume independently — ~80 lines.
PostgreSQL	PostgreSQL License	NO — pattern only	f_smgr dispatch design, smgr_immedsync/smgr_registersync shape for DelosStorageDispatch.
MariaDB/InnoDB	GPL 2+	NO — lesson only	Handler anti-pattern (218 methods), engine-owns-IO lesson, innodb_flush_method SyncPolicy analogy.
Apache Calcite	Apache 2.0	YES — directly	Table/FilterableTable interface shapes, mutable filter-list mechanism, CsvFilterableTable as reference implementation.

9

Explicit Non-Goals

Each non-goal is stated with its justification, not as a vague deferral. Many are explicitly named future lanes — the distinction is that they are not blocking current work and will be revisited with specific triggers.

No common VFS forced across providers

MariaDB source confirmed: InnoDB (os0file), MyISAM (mysys), RocksDB (RocksDB Env) each own their I/O completely and coexist without interference. DelosPageVolume is MVCC-internal; the heap adapter uses Derby's StorageFactory.

No DelosStorageDispatch before two live providers exist

PostgreSQL smgrsw[] has carried one entry and a 'we only have md.c at present' comment for decades. Build the dispatch table only after K1 proves the heap seam is clean and heap is actually live.

No io_uring by default

Jasny et al. PVLDB 2026 confirms io_uring benefits are workload-dependent: specifically beneficial at high concurrent IOPS with batched submission. A synchronized FileChannel generating one I/O per commit is not that workload. IoUringPageVolume is a named future slot gated on benchmark evidence.

No Calcite rule engine import

Calcite's Volcano planner (17 files) plus rule catalog (157 rule files, 43K lines) is the intelligence — not the contract. The FilterableTable adapter pattern is what DelosDB borrows. OptimizerImpl replacement requires a separate, dedicated effort after cost consumption (N-5) is proven.

No FilePath URL-scheme VFS layer

H2's FilePath abstraction (8+ scheme implementations) is the right reference IF encryption, split-file, or async-VFS backends become real requirements. They are not current requirements. Add DelosVolumePath only when a second storage scheme (encrypted, cloud-backed) needs to coexist.

No LeanStore-style kernel bypass by default

LeanStore bypasses the OS via SPDK to saturate multiple NVMe SSDs at tens of millions of IOPS. DelosDB's current regime is a research prototype with single-threaded synchronized FileChannel. ODirectPageVolume (F-1) is the correct intermediate step before considering full kernel bypass.

R

References

- [1] Leis, V. (2024). LeanStore: A High-Performance Storage Engine for NVMe SSDs. PVLDB 17(12). <https://doi.org/10.14778/3685800.3685915>
- [2] Alhomssi, A. & Leis, V. (2023). Scalable and Robust Snapshot Isolation for High-Performance Storage Engines. PVLDB 16(6), 1426–1438. <https://www.vldb.org/pvldb/vol16/p1426-alhomssi.pdf>
- [3] Haas, G. & Leis, V. (2023). What Modern NVMe Storage Can Do, and How to Exploit It: High-Performance I/O for High-Performance Storage Engines. PVLDB 16(9), 2090–2102. <https://doi.org/10.14778/3598581.3598584>
- [4] Jasny, M., El-Hindi, M., Ziegler, T., Leis, V. & Binnig, C. (2026). io_uring for High-Performance DBMSs: When and How to Use It. PVLDB 19(1). <https://arxiv.org/abs/2512.04859>
- [5] Houlborg, E. et al. (2026). Flexible I/O for Database Management Systems with xNVMe. CIDR 2026. <https://vldb.org/cidrdb/papers/2026/p6-houlborg.pdf>
- [6] Leis, V. & Dietrich, C. (2024). Cloud-Native Database Systems and Unikernels: Reimagining OS Abstractions for Modern Hardware. PVLDB 17(8).
- [7] OpenJDK JEP 444: Virtual Threads (Java 21). <https://openjdk.org/jeps/444>
- [8] OpenJDK JEP 491: Synchronize Virtual Threads without Pinning (Java 24). <https://openjdk.org/jeps/491>
- [9] OpenJDK JEP 454: Foreign Function and Memory API (Java 22, finalized). <https://openjdk.org/jeps/454>
- [10] H2 Database source: org.h2.mvstore.FileStore, OffHeapStore, SingleFileStore. License: MPL 2.0 / EPL 1.0. <https://github.com/h2database/h2database>
- [11] HerdDB source: herddb.storage.DataStorageManager, herddb.utils.ManagedFile, ODirectFileOutputStream, OpenFileUtils. License: Apache 2.0. <https://github.com/diennea/herddb>
- [12] MapDB source: org.mapdb.store.legacy.Volume (FileChannelVol, MappedFileVol, MemoryVol, Factory). License: Apache 2.0. <https://github.com/jankotek/mapdb>
- [13] PostgreSQL source: src/backend/storage/smgr/smgr.c, smgr.h (f_smgr struct, smgrsw[] dispatch table). <https://github.com/postgres/postgres>
- [14] MariaDB server source: sql/handler.h, storage/innobase/include/os0file.h, storage/myisam/myisamdef.h. <https://github.com/MariaDB/server>
- [15] Apache Calcite 1.42.0 source: core/src/main/java/org/apache/calcite/schema/ Table.java, FilterableTable.java, ModifiableTable.java. License: Apache 2.0. <https://github.com/apache/calcite>
- [16] Apache Derby / DelosDB: delosdb-storage-mvcc, delosdb-engine-kernel. License: Apache 2.0. <https://github.com/ggeorg/delosdb>